

软件架构文档化

北京交通大学软件学院

严禁复制



目录



01. 架构文档的意义

02. 架构文档的编写方法

03. 架构描述语言

04. 作业

严禁复制

内容

- 架构文档的意义
 - 文档目标
 - 文档的干系人
 - 视图
 - 敏捷项目的文档化
 - 指导原则
- 文档编写方法
 - Kuchten 4+1 视图
 - 视图和视图之外
 - C4模型
 - Arc42

严禁复制



01 复制 软件架构文档的意义

代码只是架构的一部分

- 代码没有讲述完整的故事
- 代码没有回答的问题
 - 该软件如何适应基础架构?
 - **为什么**选择这些技术?
 - 系统的整体结构是什么?
 - 组件在运行时部署在哪里?
 - 组件之间如何通信?
 - 如何**添加**新功能?
 - 使用了哪些模式和原则?
 - 与其他系统的协作?
 - 如何实现安全性/可扩展性/.....等?

严禁复制

文档目标

- 主要目标：传达结构
 - 理解整体框架
- 创建共同愿景：团队与利益相关者
 - 统一术语
- 描述软件是什么以及如何构建
 - 聚焦关于新功能的技术讨论
 - 提供导航源代码的地图
- 论证设计决策
- 帮助新加入团队的开发者

禁止复制

文档的需求

- 让不同利益相关者都能理解
 - 技术与非技术利益相关者
- 反映真实情况
 - 注意模型与代码之间的差距
- 快速开发并适应变化
 - 适应敏捷项目
 - 演进式架构

严禁复制

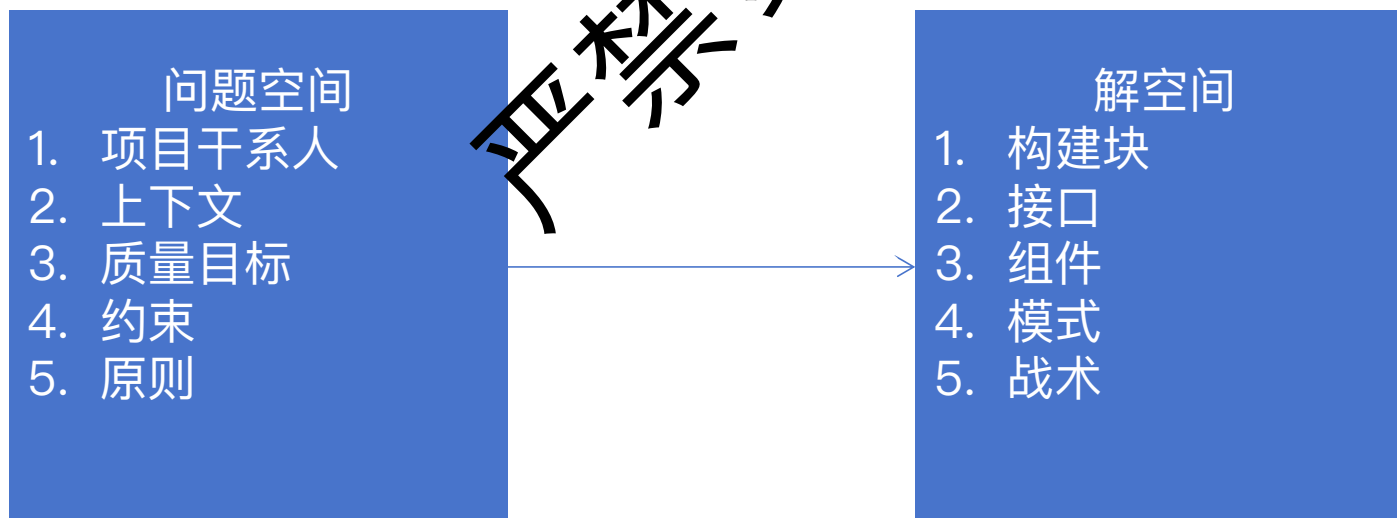
好文档的原则

- 从读者的角度编写文档
 - 找到读者并了解他们的期望
- 避免不必要的重复（DRY原则）
- 避免歧义
 - 解释符号（或使用标准符号）
 - 对于图表，使用图例
- 使用标准的组织方式或模板
 - 必要时添加TBD/待办事项
 - 组织以便于参考/链接
- 记录决策依据
- 保持文档更新

严禁复制

问题空间 VS 解空间

- 软件架构 = 从问题到解决方案的路径
 - 理解问题
 - 设计解决方案
 - 记录所提方案的决策依据
 - 记录不同的设计备选方案



视图 & 视角

- 软件架构是一个复杂的实体
 - 无法用单一维度描述
 - 它需要针对不同利益相关者的多种视图
- 视图 = 针对特定关注点的系统表示方式
 - 不同视图服务于不同目标和用途
- 视点 = 用于构建视图的模式、模板和惯例的集合
 - 示例：结构视图、行为视图、部署视图

视图是你看到的
视角是你从哪里看过去

文档中如何描述视图

1. 引言

1. 视图的文字描述

2. 图表

1. 添加描述性标题（需包含所描述的结构）
2. 创建图例说明符号含义
 1. 不要忘记解释线条/箭头的含义

3. 元素清单及其职责

1. 使用描述性命名
2. 定义术语（包含术语表）

4. 设计依据

严禁复制

文档中如何描述视图

- 力求一致性和简洁性
- 保持元素的一致性
 - 颜色、形状、箭头等
 - 如果使用配色方案，请始终遵循
 - 检查不同视图中的命名是否一致
 - 记录可能的不一致之处
- 避免过多细节
 - 记住米勒定律
 - 普通人一次只能记住7 (± 2) 个元素

严禁复制

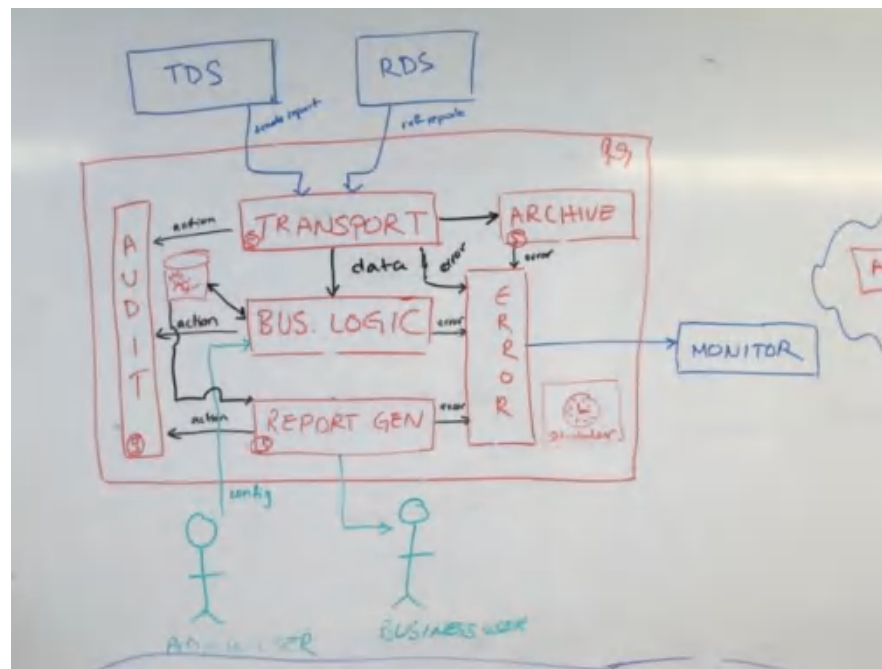
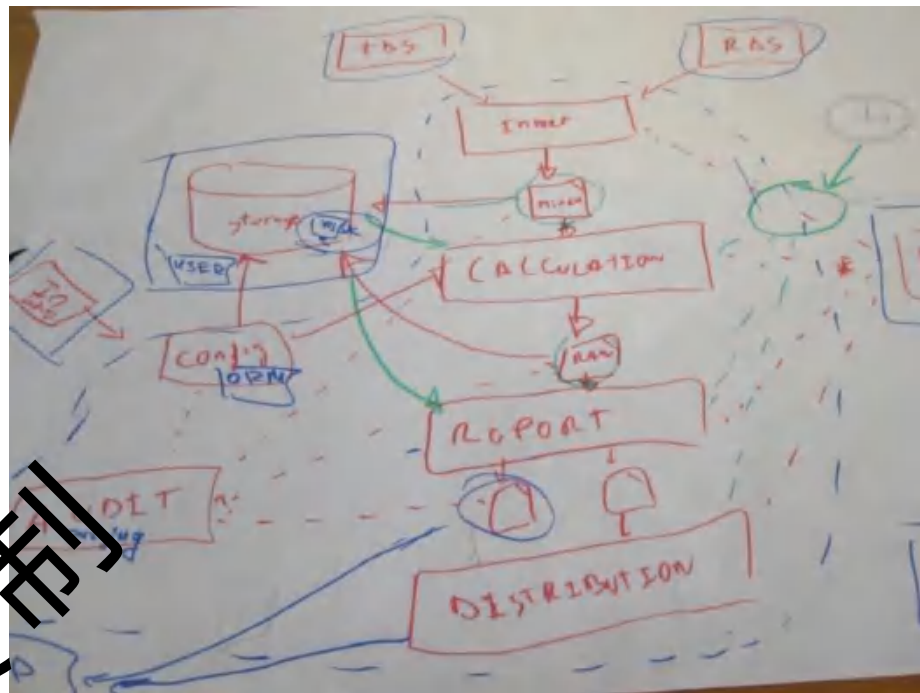
架构设计的画图工具

- 手绘草图 (Sketches)
- 绘图工具 (Drawing Tools for Diagrams)
 - Visio、Lucidchart、Draw.io
- 基于文本的图表工具 (Text-Based Diagramming Tools)
 - PlantUML、Mermaid
- 建模工具 (Modeling Tools)
 - Enterprise Architect、MagicDraw
- 逆向工程模型 (Reverse-Engineering the Model)
- 架构描述语言 (Architecture Description Languages)
 - UML、SysML、ArchiMate

禁止转载

手绘草图

- 大多数人会从在纸上或白板上绘制草图开始
 - 这是协作和交换想法的绝佳方式
 - 通常这些草图的寿命较短
- 但迟早需要被记录下来
 - 简单的方法：拍照
 - 随后将其转换为图表或模型



架构绘图工具

- 桌面
 - Microsoft Visio, Omnigraffle, SimpleDiagrams
- WEB
 - draw.io, gliffy, LucidChart
- 通用绘图工具
 - Word, Powerpoint, Keynote

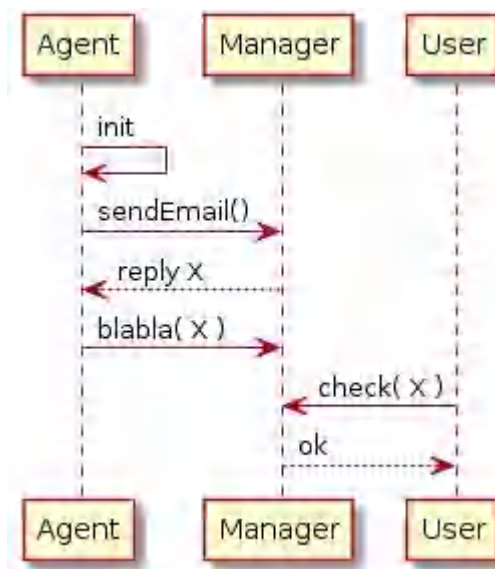
严禁复制

文本绘图工具

- 通常是基于UML的
 - plantuml.com
 - www.mermaidchart.com

```
@startuml
Agent -> Agent : init
Agent -> Manager : sendEmail()
Agent <-- Manager : reply X
Agent -> Manager : blabla( X )
User -> Manager : check( X )
User <-- Manager : ok
@enduml
```

严禁复制



建模工具

- 允许创建软件系统的模型
- 从模型生成可视化表示
- 可选工具：
 - Sparx Enterprise Architect、Visual Paradigm、Archi、StarUML、ArgoUML、Modelio 等
- 通常支持多种符号
 - 如 UML、SysML、BPMN、ArchiMate
- 适用于前期设计
- 对重构和重命名组件等操作非常有用

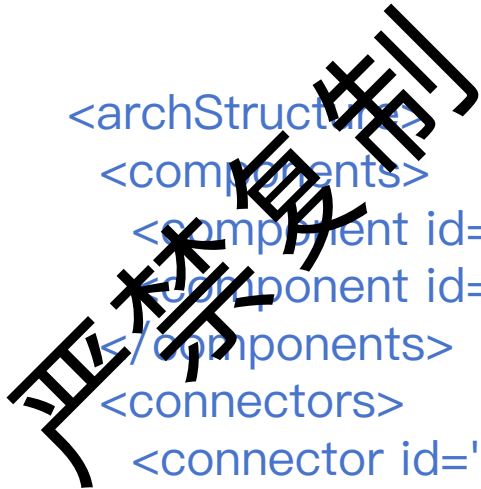
逆向工程建模

- 一些先前提到的建模工具支持此功能
- 静态分析工具：
 - Structure101、NDepend、Lattix、Sonargraph 等
- 基于现有代码创建模型
 - 有助于可视化现有代码库
- 问题：
 - 生成的图表往往包含过多细节
 - 难以看清整体架构

严禁复制

架构描述语言：ADLs

- 正式定义系统架构
 - 创建文本描述而不是图表
 - 正式规范
 - 描述结构和行为
- 主要用在学术圈
 - 工业界不常用
- 一些例子
 - xArch/xADL
 - ACME
 - AADL



```
<archStructure>
  <components>
    <component id="Component1" type="Service"/>
    <component id="Component2" type="Database"/>
  </components>
  <connectors>
    <connector id="Connector1" from="Component1"
to="Component2" type="RPC"/>
  </connectors>
</archStructure>
```

ACME

- 描述、分析和交换软件系统的架构信息
- 目标：
 - 架构设计：
 - 用于描述和记录软件系统的架构设计。
 - 架构分析：
 - 通过工具对架构进行静态或动态分析，发现潜在问题。
 - 架构交换：
 - 作为中间表示形式，支持不同 ADL 之间的模型转换和集成。
 - 领域特定扩展：
 - 通过扩展 ACME，支持特定领域（如嵌入式系统、分布式系统）的架构描述。

```
System simpleSystem = {  
  Component client = {  
    Port sendRequest;  
  };  
  Component server = {  
    Port receiveRequest;  
  };  
  Connector rpc = {  
    Roles {caller, callee};  
  };  
  Attachments {  
    client.sendRequest to rpc.caller;  
    server.receiveRequest to rpc.callee;  
  };  
};
```

AADL

- 嵌入式实时系统的设计、分析和验证
- 组件：
 - 软件组件（如进程、线程、数据）
 - 硬件组件（如处理器、内存、设备）
 - 每个组件都有明确的接口和属性。
- 连接：
 - 组件之间通过连接进行交互，
 - 例如数据流、控制流或物理连接。
- 属性：
 - 组件和连接可以附加属性，用于描述其行为或约束，
 - 例如执行时间、内存占用、通信带宽等。

```
thread implementation SensorThread.impl  
end SensorThread.impl;
```

```
process implementation SensorProcess.impl  
subcomponents  
  sensorThread: thread SensorThread.impl;  
end SensorProcess.impl;
```

```
processor implementation MainProcessor.impl  
end MainProcessor.impl;
```

```
system implementation SensorSystem.impl  
subcomponents  
  sensorProcess: process SensorProcess.impl;  
  mainProcessor: processor MainProcessor.impl;  
connections  
  connect sensorProcess to mainProcessor;  
end SensorSystem.impl;
```

严禁复制



02 复制 软件架构文档的写法

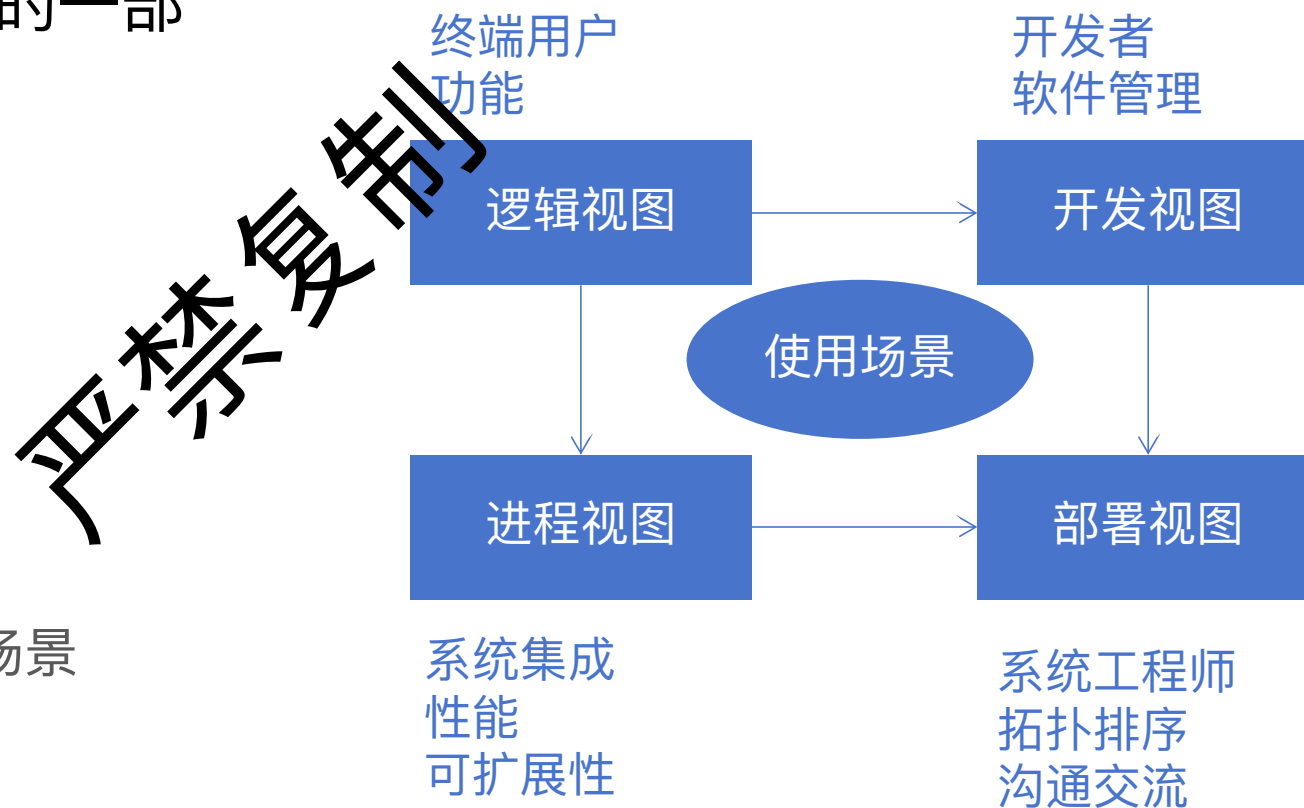
软件架构模板

- Kruchten4+1 views
- Views & beyond
- C4 model
- Arc42 templates

严禁复制

Kruchten 4+1 views

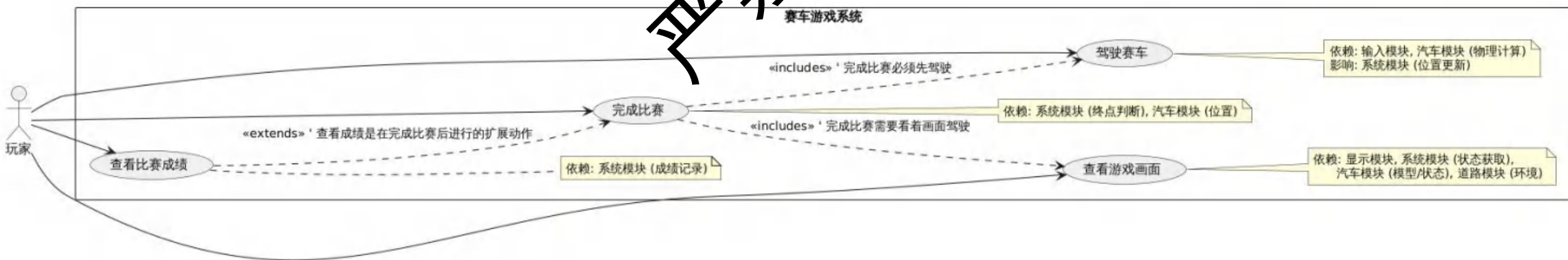
- 作为Rational Unified Process (RUP, 统一软件开发过程) 的一部分被采纳
- 5个并行的视图
 - 逻辑视图：系统的功能
 - 开发视图：模块、层次结构等
 - 进程视图：执行单元、并发性等
 - 物理视图：基础设施与部署拓扑
 - (+1) 场景视图：选定的用例或场景



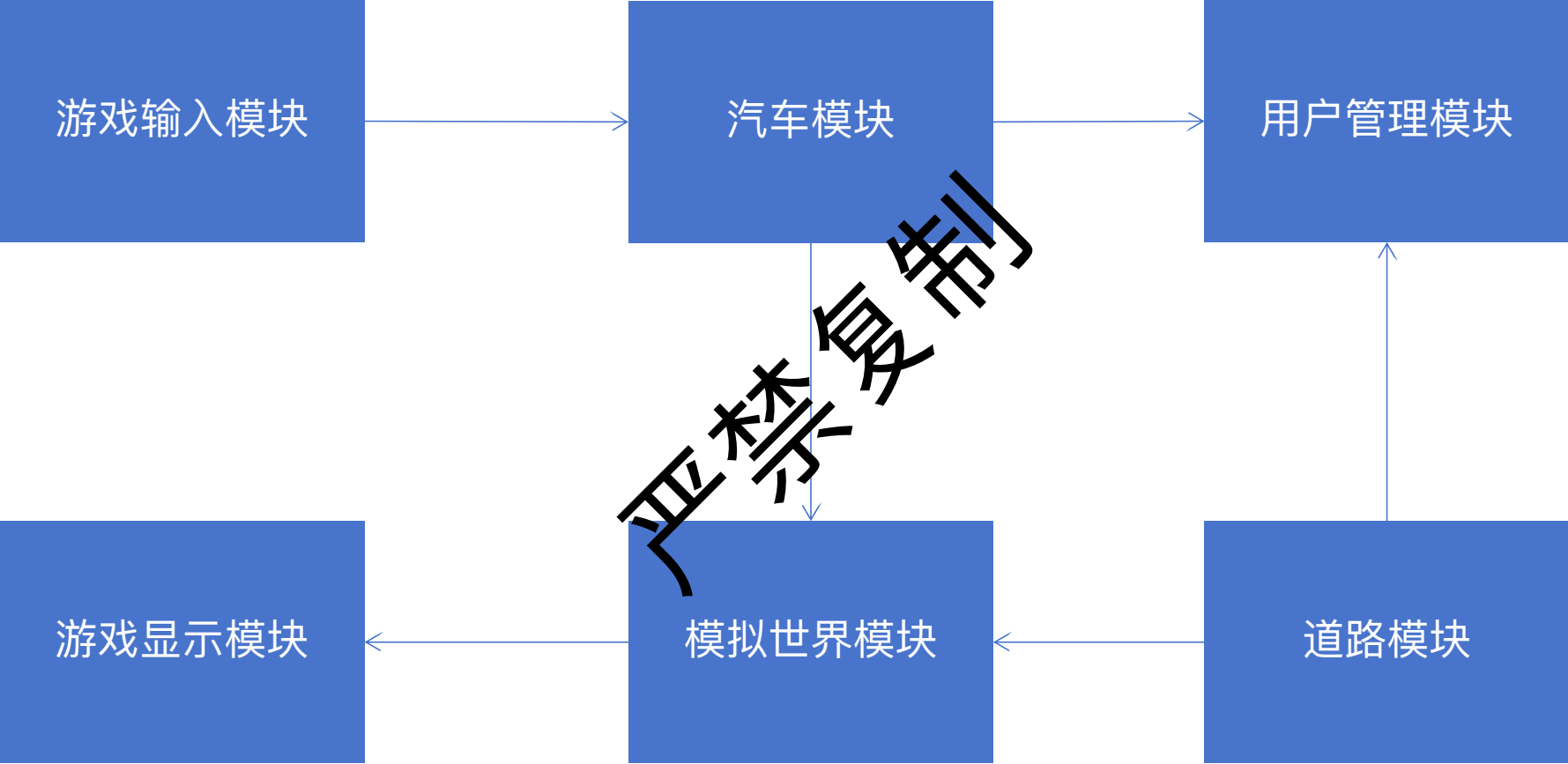
使用场景:赛车游戏系统

- 用例 (Use Cases):

- 驾驶赛车 (Drive Car): 玩家控制赛车的核心动作, 包括加速、刹车、转向。
- 查看游戏画面 (View Game Screen): 玩家通过显示模块观察赛车视角、道路情况和游戏界面。
- 完成比赛 (Complete Race): 玩家驾驶赛车到达终点线。
- 查看比赛成绩 (View Race Results): 比赛结束后, 玩家查看所用时间等成绩信息。

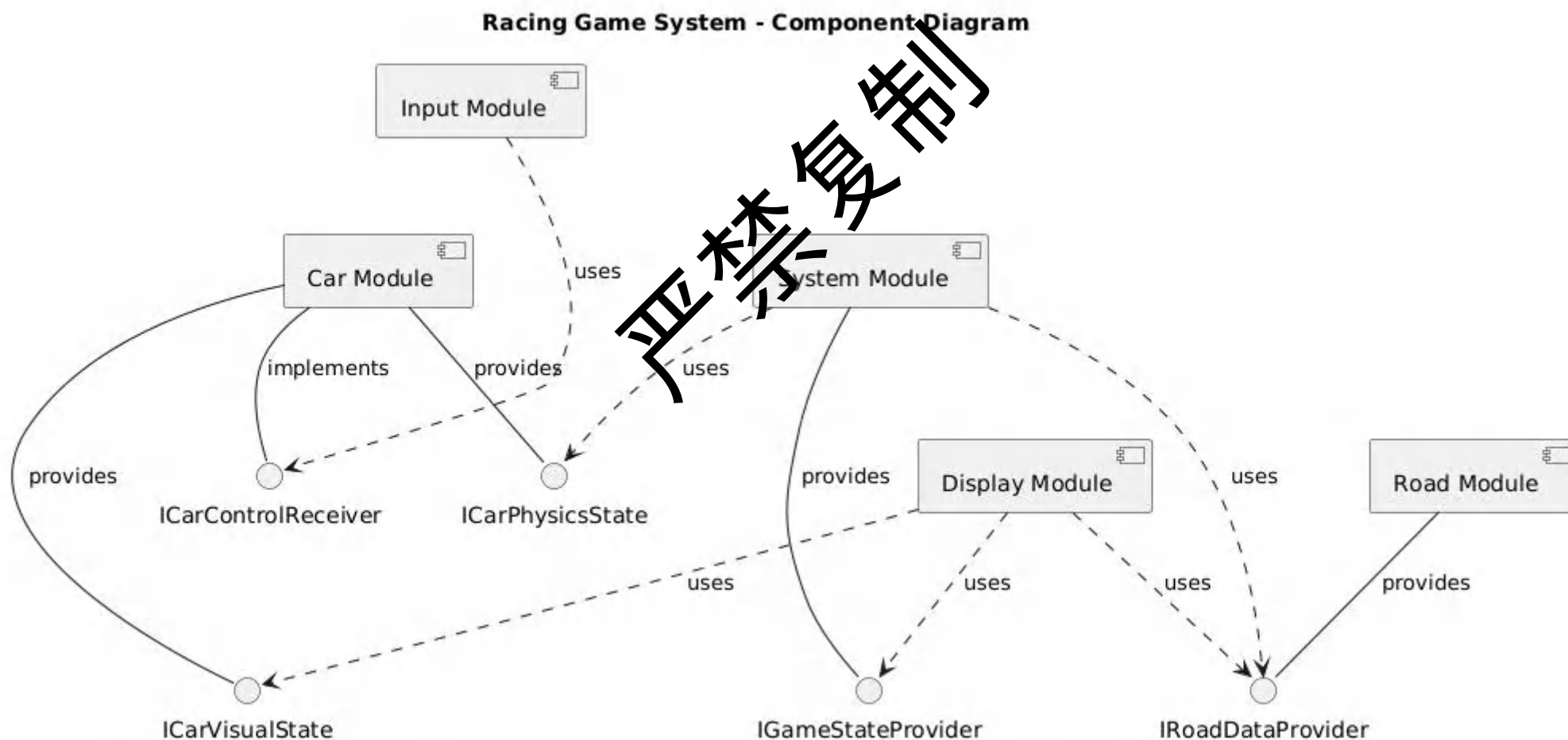


逻辑视图（模块图）：赛车游戏系统

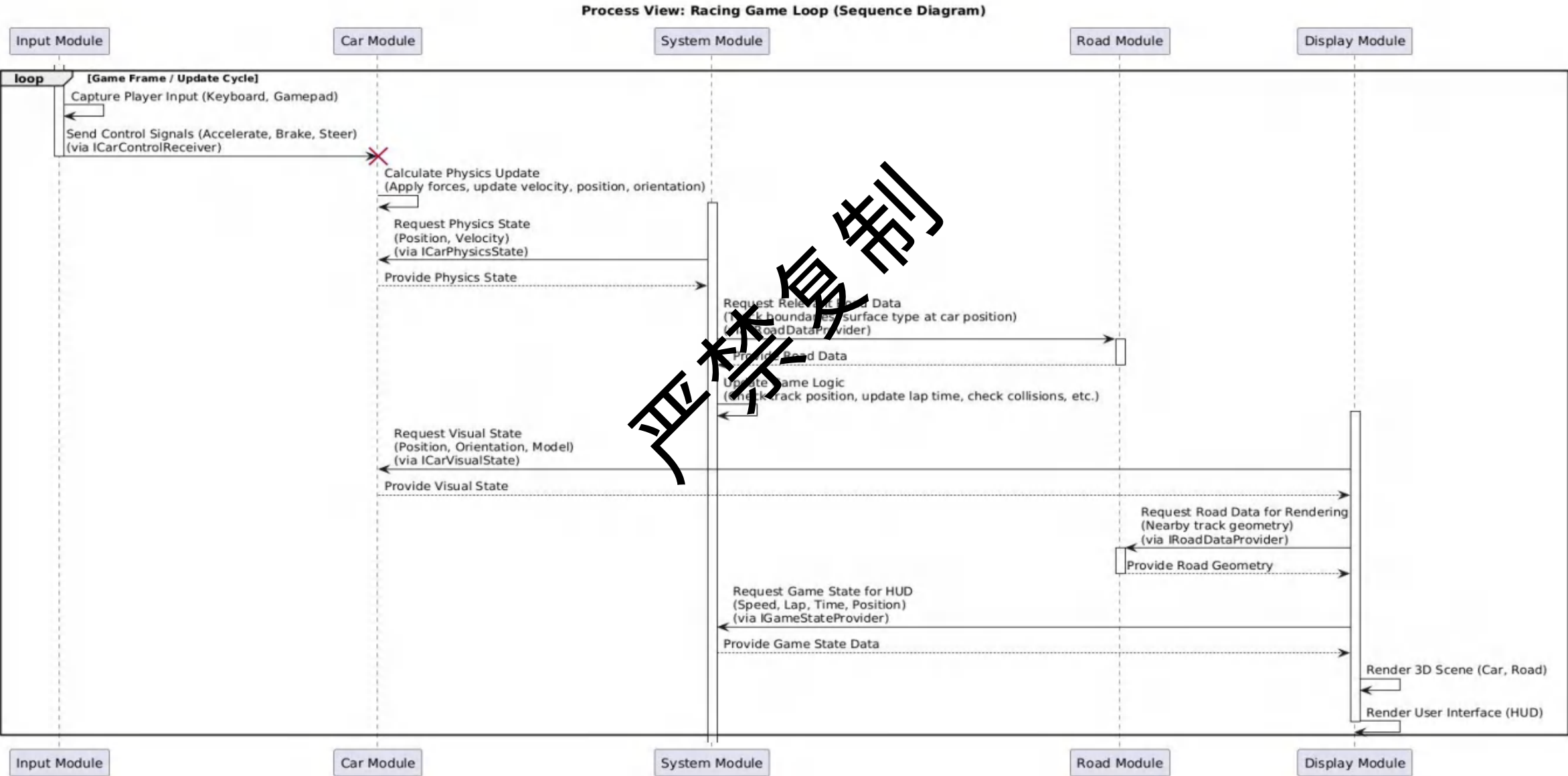


开发视图（组件图）：赛车游戏系统

- 该系统被设计为一个模块化的结构，包含五个核心组件，每个组件负责特定的功能，并通过定义的接口进行交互。



进程图：赛车游戏系统



4+1视图的架构文档结果

Title Page

Change History

Table of Contents

List of Figures

1. Scope

2. References

3. Software Architecture

4. Architectural Goals & Constraints

5. Logical Architecture

6. Process Architecture

7. Development Architecture

8. Physical Architecture

9. Scenarios

10. Size and Performance

11. Quality

Appendices

A. Acronyms and Abbreviations

B. Definitions

C. Design Principles

严禁复制

4+1视图的优势

- 关注点分离 (Separation of Concerns):
 - 逻辑结构、运行时行为、开发组织、物理部署、功能需求
- 满足不同利益相关者的需求 (Addresses Stakeholder Needs):
 - 逻辑视图 (Logical View): 主要面向设计师和开发者，关注系统的功能和类/组件结构。
 - 过程视图 (Process View): 主要面向集成者和性能工程师，关注系统的并发性、性能、可伸缩性和容错性。
 - 开发视图 (Development View): 主要面向开发者和配置管理者，关注代码的组织、模块划分和构建过程。
 - 物理视图 (Physical View): 主要面向系统工程师和运维人员，关注系统如何部署到硬件节点上。
 - 场景/用例视图 (Scenario/Use Case View – the “+1”): 主要面向所有利益相关者（特别是分析师、设计师、测试人员），用于驱动架构设计并验证其他四个视图的一致性和完整性。
- 提高沟通效率 (Improved Communication): 减少了误解。
- 覆盖全面 (Comprehensive Coverage): 从高层功能到底层部署，。
- 驱动和验证架构 (Architecture Validation): “+1” 的用例视图起着核心作用。
- 有助于管理复杂性 (Manages Complexity):。

4+1视图的不足

- 开销较大 (Significant Overhead);
- 视图间一致性维护困难 (Consistency Challenges);
- 学习曲线 (Learning Curve): 需要一定的学习和经验;
- 可能过于僵化 (Potential Rigidity): 设计过程变得僵化, 不敏捷;
- 工具依赖性 (Tool Dependency);
- 侧重于技术结构, 可能忽略业务架构, 业务战略和业务流程;
- 不适用于特定系统:例如, 纯数据处理管道、非常小的应用程序;

思考题

- 你会使用4+1 视图吗?
- A: 会
- B: 不会

严禁复制

超越视图的文档方法 (1)

- “Views and Beyond” 是一种强调 实用性 和 完整性 的架构文档化方法论。它主张：
 - 使用 精心选择的多个视图 来结构化地描述架构的不同方面。
 - 超越 这些视图，补充：
 - 关键的上下文信息、
 - 设计原理、
 - 决策依据、
 - 质量属性考虑，
 - 目标：
 - 使得架构文档不仅能描述系统结构，更能解释设计背后的思考过程，从而更好地支持系统的开发、部署、维护和演进。

严禁复制

超越视图的文档方法-beyond (2)

- 引言
 - 系统概述，文档目标
- 架构驱动因素
 - 业务目标，功能需求，质量属性，约束条件
- 系统上下文与范围
 - 与其它系统如何协作，端到端的使用场景
- 架构原理与决策
 - 架构原则 (Guiding Principles)
 - 重要设计决策 (Significant Design Decisions / ADRs)
- 接口文档
 - 对外提供的接口，内部使用的接口
- 风险与缓解措施
 - 潜在风险，以及如何缓解

严禁复制

超越视图的文档方法 (3)

- 更全面的系统理解
 - 使用多种视图
 - 补充关键信息
- 改善沟通效率
 - 针对不同的利益相关者提供独立的内容
 - 记录决策北京

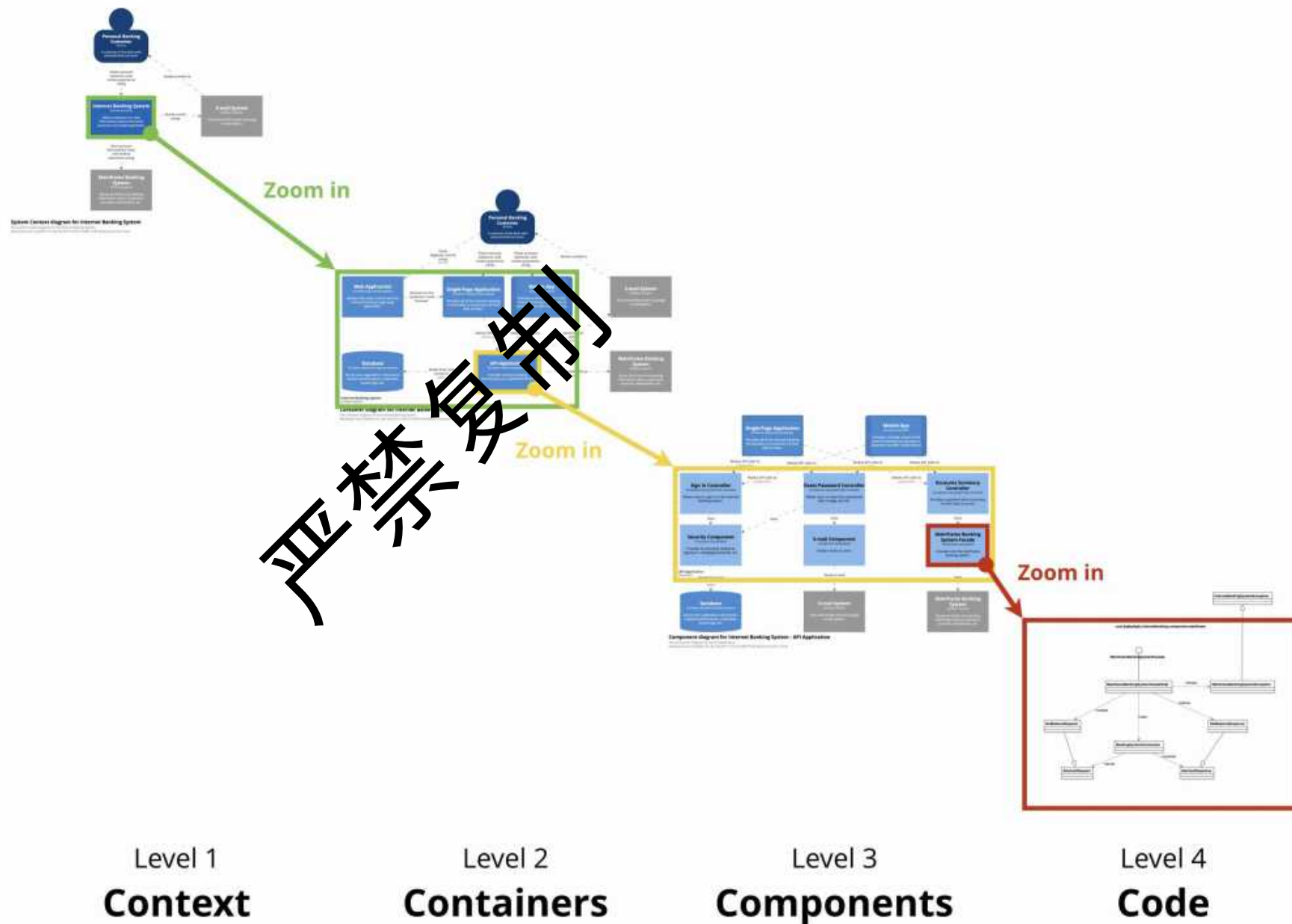
严禁复制

C4 model

- 定义
 - 上下文context
 - 容器container diagram
 - 组件 component diagram
 - 代码code

严禁复制

C4 Model



C4 model的解释

- C1: Context (系统上下文)
 - 回答“系统是什么？”、“谁使用它？”以及“它与哪些其他系统交互？”的问题
 - 用户/角色: 赛车手 (Driver)
- C2: Containers (容器)
 - 回答“系统由哪些主要部分组成？”、“它们的技术选型是什么？”以及“它们如何通信？”的问题。通常面向软件开发者、运维人员和架构师
- C3: Components (组件)
 - 回答“这个容器（如 Web 应用或服务）内部是如何组织的？”、“它的主要职责模块有哪些？”以及“这些模块如何协作？”的问题。主要面向负责该容器开发的软件工程师。
- C4: Code (代码 / 类)
 - 这是最详细的层次，会“放大”到某一个 组件 内部，展示其代码层面的实现细节，例如类图、接口、函数签名等。

C4 Model 对比 4+1 视图

- 优势：

- 简单性和易学性：只有4个level
- 沟通效率：C1业务人员，C2运维 C3 研发团队 C4研发人员
- 层次化和导航性：容易找到感兴趣的组件
- 与现代软件开发实践的契合度：对微服务友好
- 图表一致性和维护：C3组件图限制在C2容器之内
- 灵活性和非强制性：每层的图表类型不限制

- 劣势：

- 4+1 视图模型在明确区分系统的不同方面（如功能分解、并发性、开发组织、物理部署）
- 深入分析特定非功能性需求（如性能、可靠性）或复杂部署场景

Arc42 <https://arc42.org>

- 软件系统文档结构
- 目标：清晰，简单，有效
- 多种模板
 - AsciiDoc
 - Word (docx)——适合本地编辑
 - Markdown——适合在浏览器上编辑
 - LaTeX
 - ReStructuredText
 - Confluence

禁止转载

Arc42 概述

问题

1. 概述和目标
2. 约束
3. 上下文和范围

解决方案

5. 构建block view
6. Runtime view
7. deployment view
8. Cross cutting concepts
9. 架构decision
10. 质量属性
11. 风险和技术债
12. 整体词汇表

严禁复制

1 – 介绍 和 目标

- 简单描述

- 需求（关键需求）
- 质量目标（top 3–5）
- 项目相关人



1-如何选择质量属性（继续）

- 质量属性研讨会
 - 邀请利益相关者参与，以确定质量属性的优先级
- 区分以下几类属性可能会有所帮助：
 - 运行时质量属性
 - 性能、安全性、可用性、易用性，等等
 - 非运行时质量属性
 - 可修改性、可移植性、可重用性、可测试性
 - 业务质量属性
 - 成本、进度、上市时间，等等

禁止转载

1-如何选择质量属性

● 产品质量

功能适用性
性能效率
兼容性
易用性
可靠性
安全性
可维护性
可移植性

● 使用质量

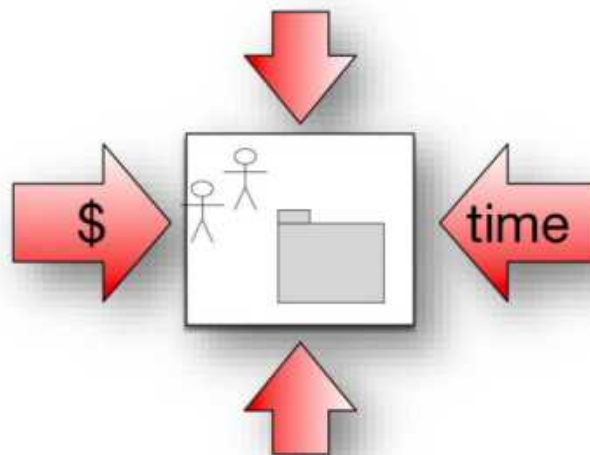
有效性
效率
满意度
风险规避性
情境覆盖性

严禁复制

2 – 约束

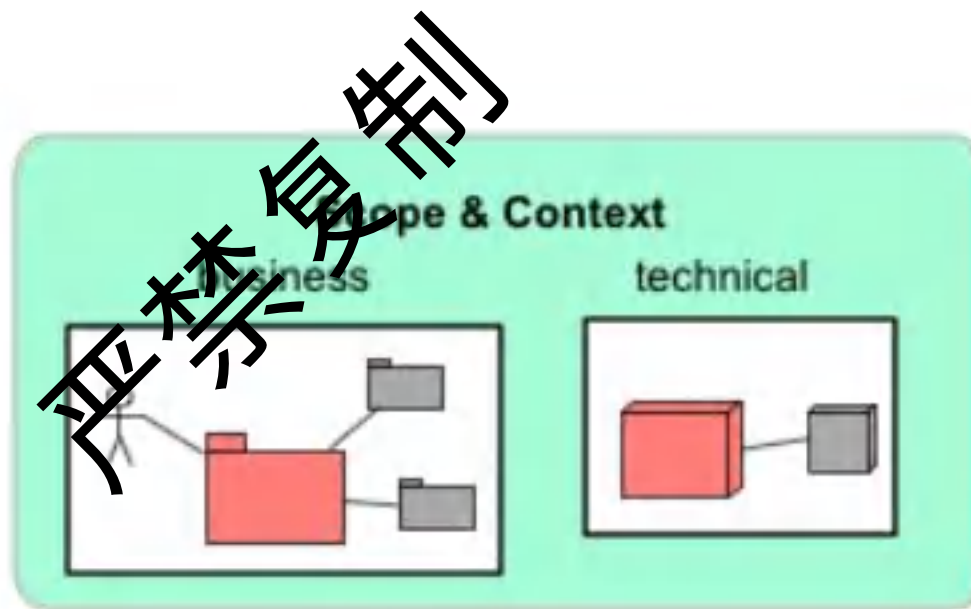
- 对团队的决策构成约束的任何因素
- 这些约束有时来自组织层面
- 也可能是已经做出的决策
- 格式：采用带解释说明的表格
 - 可以将这些约束划分为不同类别，例如：组织性约束、技术性约束等

Constraint	Explanation
...	...



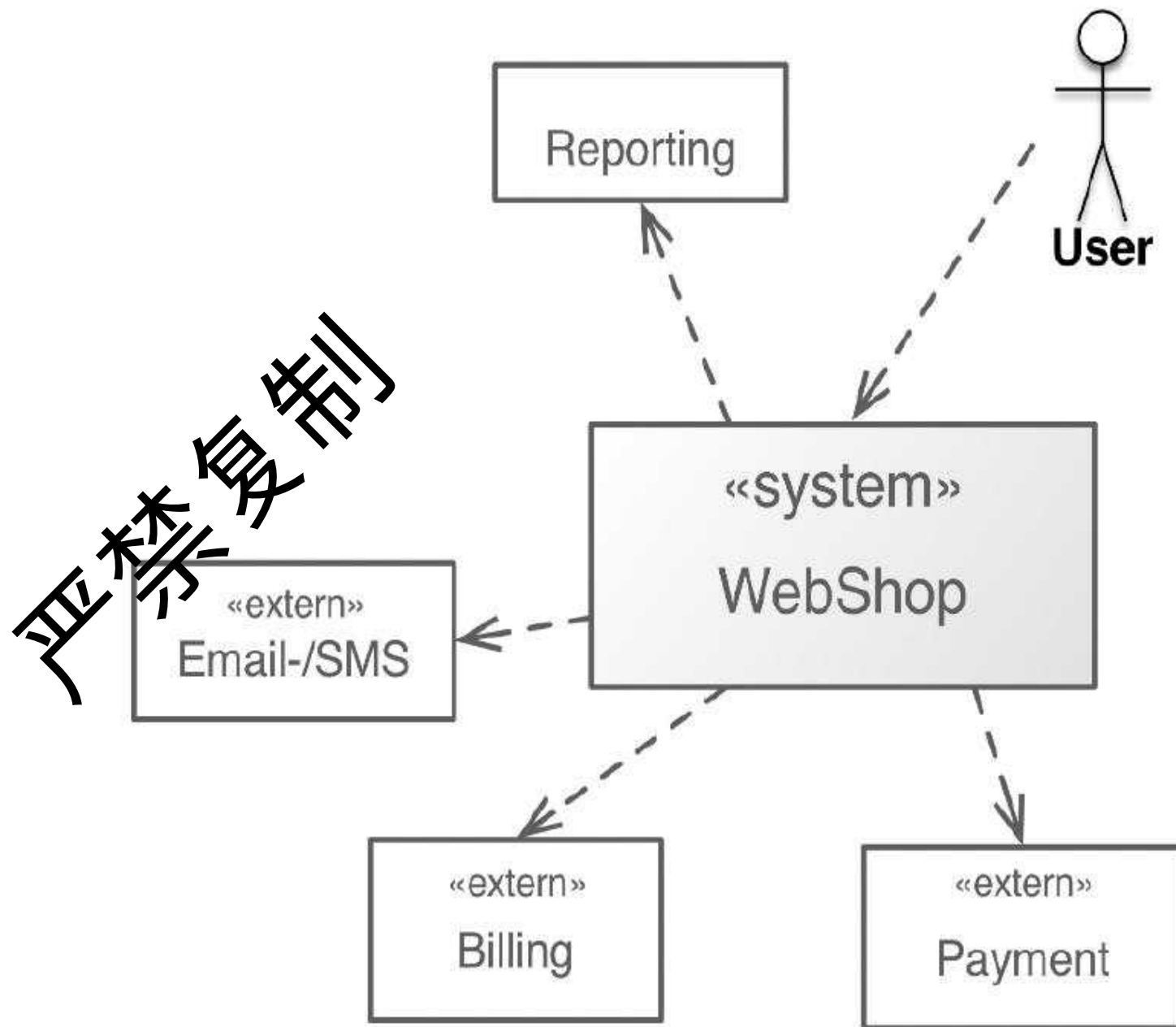
3 – 上下文和系统边界

- 划定系统与外部合作伙伴的边界
 - 相邻的用户和系统
- 定义/指定外部接口
- 业务和技术视角



3-1 业务上下文

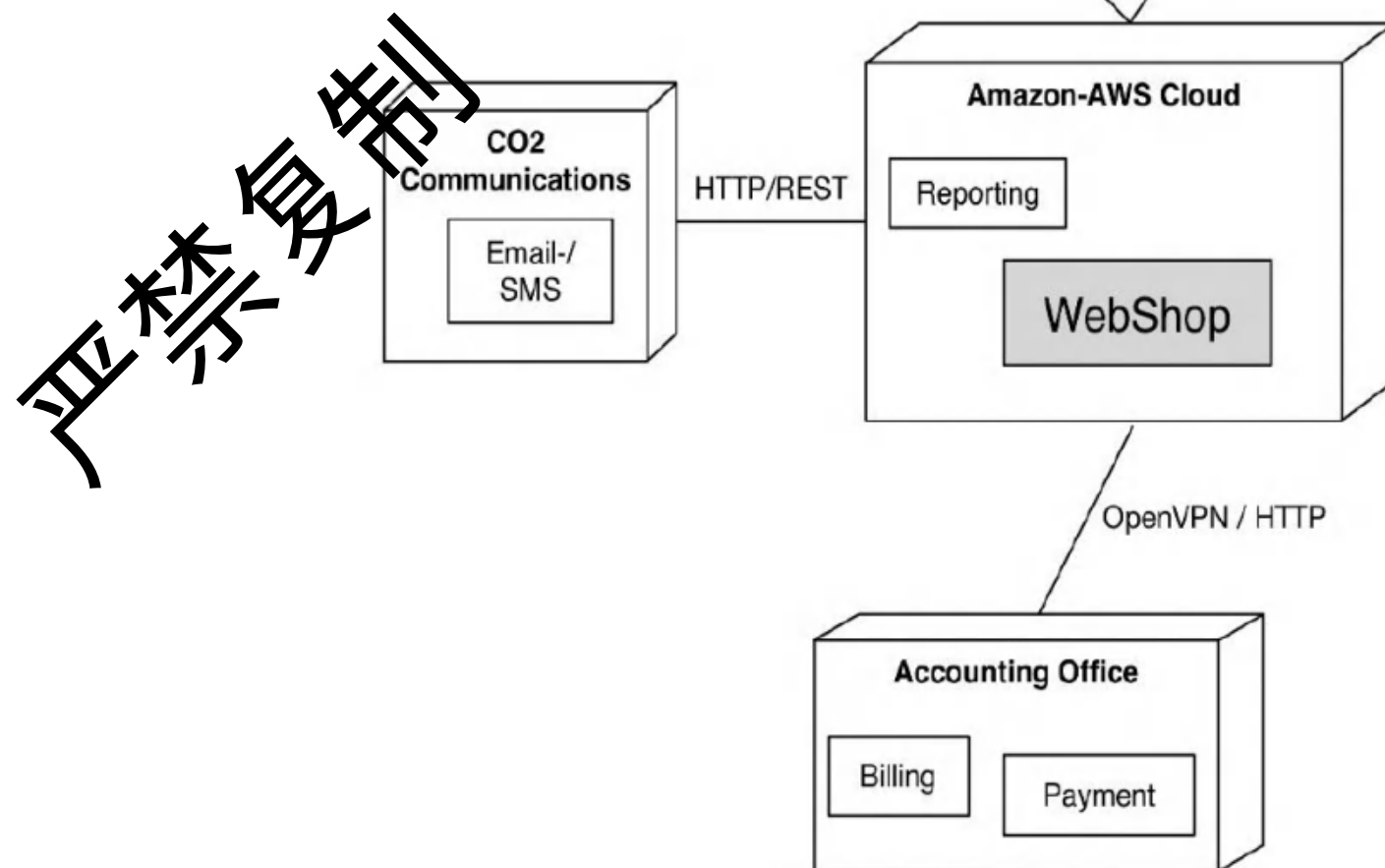
- 明确系统涉及的所有合作方
- 格式：图或表格
 - 在图中将系统展示为黑盒
- 可选：外部接口说明
- 系统的业务上下文是必须要写的



3-2 技术上下文

- 描述系统与环境之间的技术接口
- 格式：图
 - 通常会使用部署图
- 例子：html校验器
 - 快速发现基础错误
 - 确保基本结构完整

Technical Context



4- 解决方案的设计策略

- 基本决策与策略总结
- 可以包含：
 - 技术选型
 - 顶层分解：EDA，微服务
 - 实现关键质量目标的方法
 - 团队相关的决策：敏捷过程
- 格式： 简短文本描述
 - 关键决策的说明应保持简短

严禁复制

5 – 模块视图

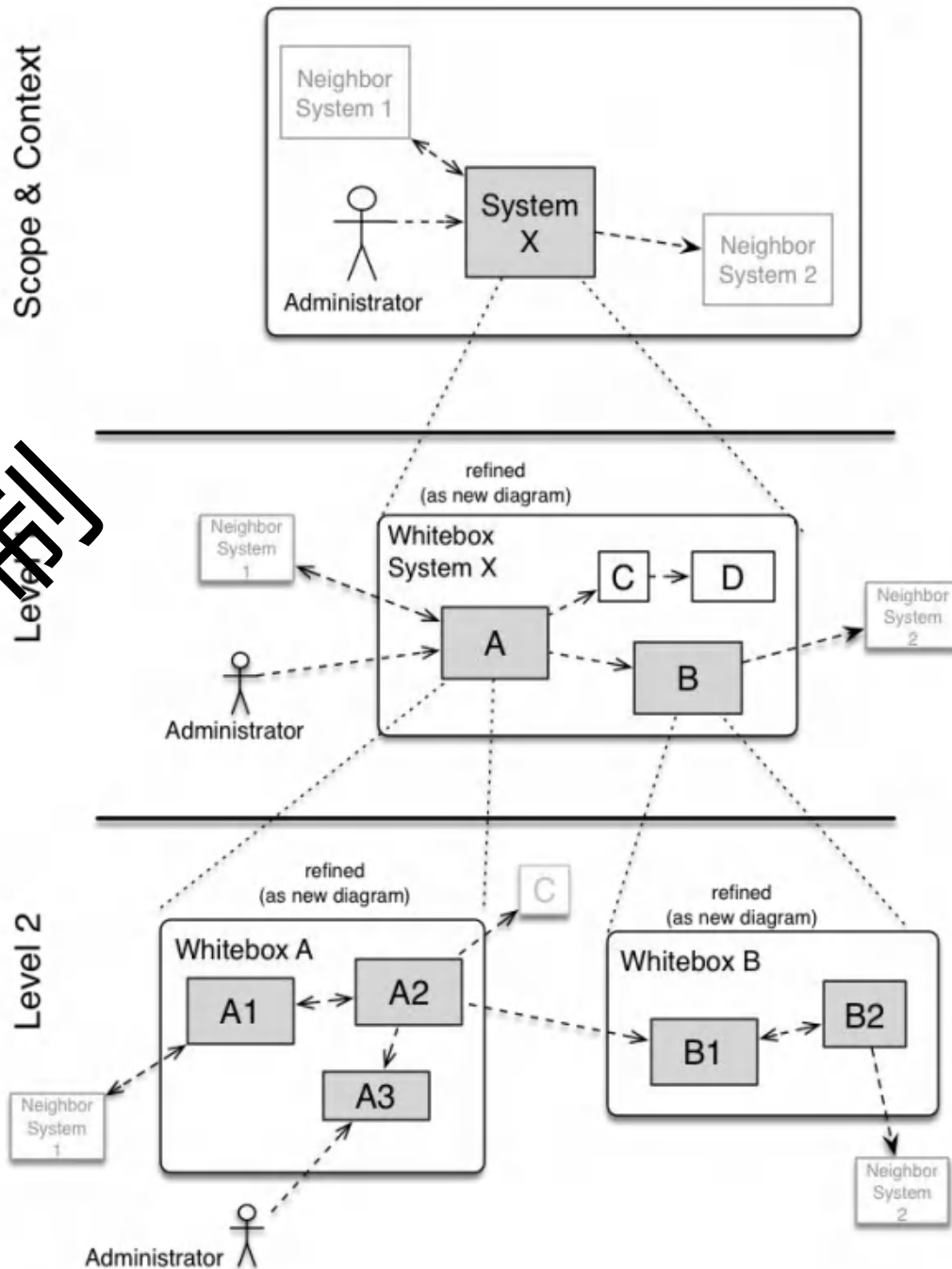
- 系统的静态分解

- 高层结构: Subsystems, Modules, Components
- 架构模式: Layers, Tiers, Partitions
- 代码组织/抽象: Packages, Classes, Interfaces
- 可重用/依赖: Libraries, Frameworks
- 实现细节: Functions/Operations, Data Structures, Macros

- 白盒包含黑盒的层次结构

- 格式:

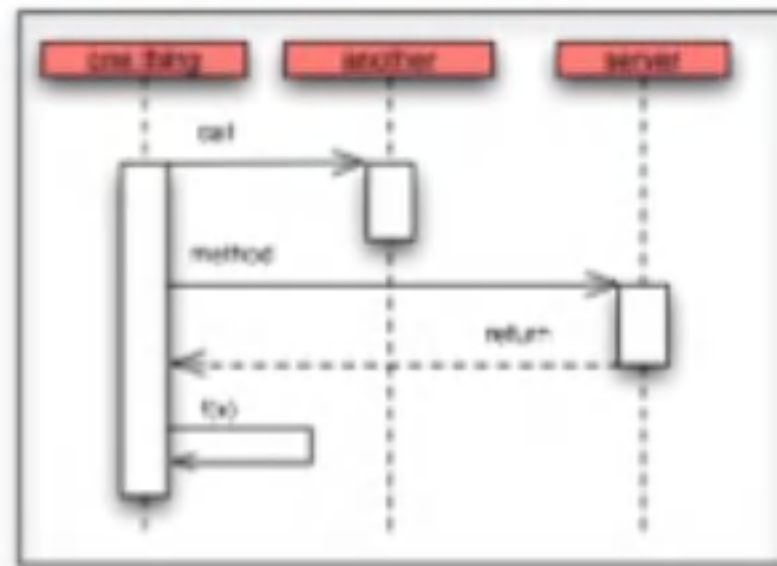
- 从整体概览图开始
- 分解为其他图表
 - 通常: UML 组件图
- 类似于C4



6 – 运行时视图

- 将构建块的行为描述为场景
 - 重要的用例或特性
 - 关键外部接口处的交互
 - 错误和异常行为
- 格式：多种表示法
 - 自然语言（步骤列表）
 - UML 序列图
 - 流程图
 - BPMN或者event链
 - 状态机

严禁复制



7 – 部署图

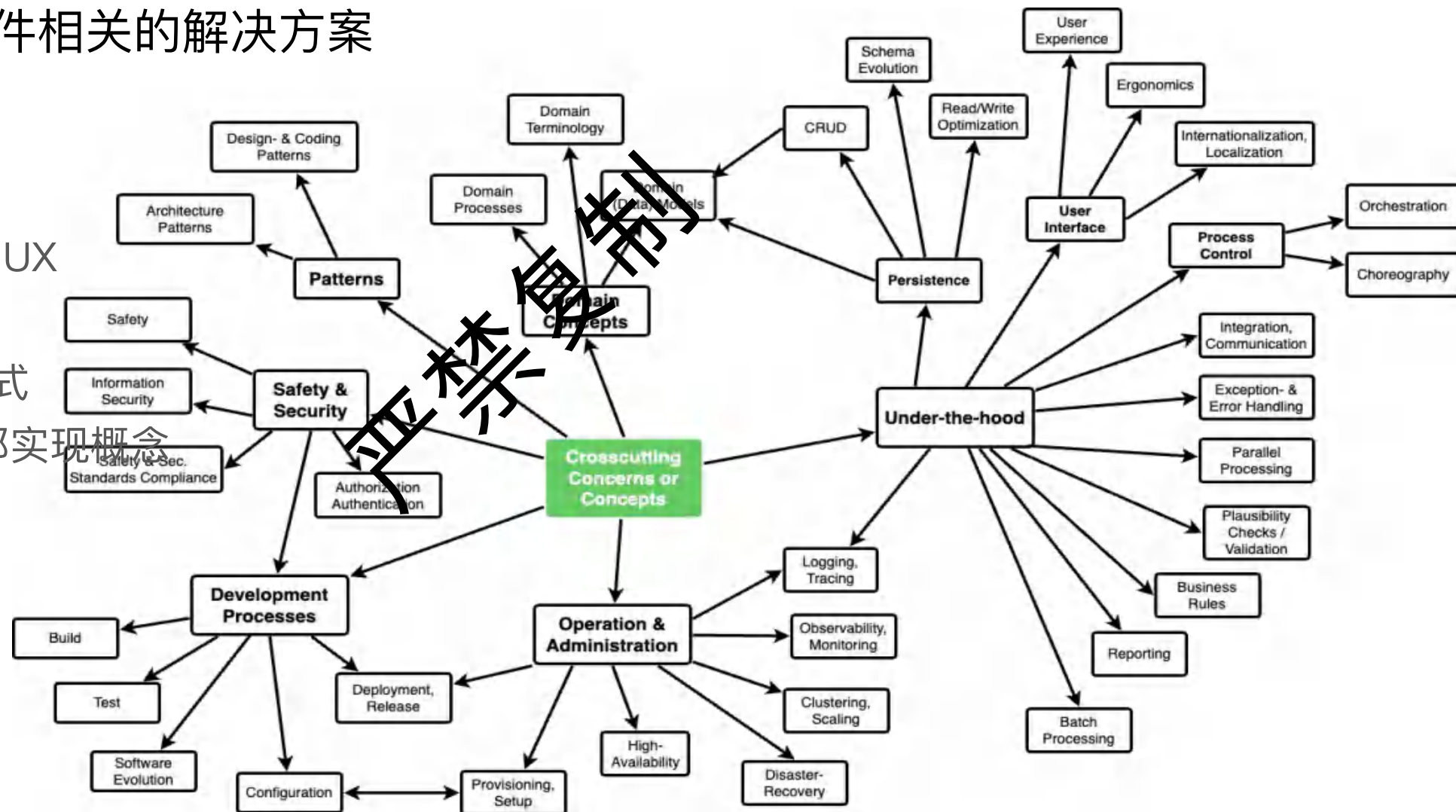
- 技术基础设施（包含环境、计算机、处理器、拓扑结构）
 - 软件构建块到基础设施的映射
 - 格式：
 - 通常：UML 部署图
 - 添加映射表
- 例子： 谷歌在全球运营着大量（具体数量不公开，但至少数十个）高度优化、节能的大型数据中心。这些数据中心战略性地分布在北美、南美、欧洲、亚洲等关键地理位置，靠近主要用户群体和互联网交换点。

8 – 切面的关键概念

- 与系统多个部件相关的解决方案

- 主题包括：

- 领域概念
- 用户体验概念 UX
- 安全保证概念
- 架构与设计模式
- 底层机制/内部实现概念
- 开发概念
- 运维概念



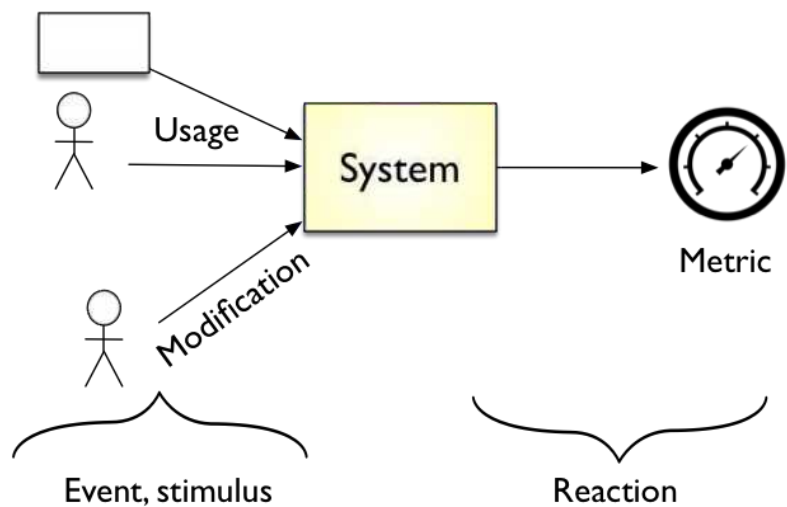
9 – 架构决策

- 重要的、成本高昂的、关键的、大规模的或具有高风险的架构决策
- 包含做出这些决策的理由/依据
- 格式：
 - 按重要性排序的列表或表格
 - 对于重要决策，采用架构决策记录 (Architecture Decision Record, ADR)
 - 每个重要的一个决定都需要单独的一小节

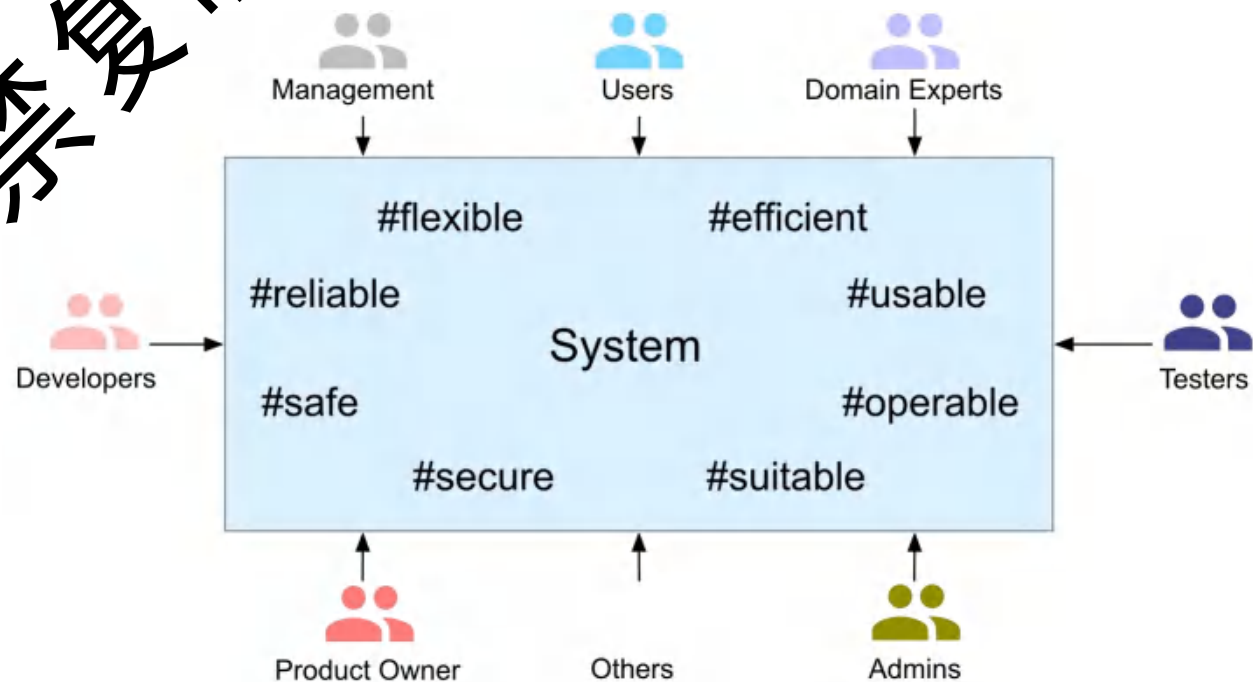
章节	描述
标题	决策应有一个合适的名称。例如：“ADR 42：针对多种检查算法的策略模式”
背景	描述当前的情况，包括技术、政治、社会和项目等方面。
决策	我们最终做出的选择是什么（用于回应“背景”部分描述的各种因素）。
状态	决策当前的状态。例如：“提议的”；“已接受”；“已废弃”或“已被取代”
影响	该决策会导致或可能引发的结果。不仅仅是“正面”影响。

10 – 质量需求

- 质量需求的使用场景
 - 质量树提供高层概述视图
 - 最重要的质量目标在第一节描述
 - 第10小节描述相对不重要的质量属性



严禁复制

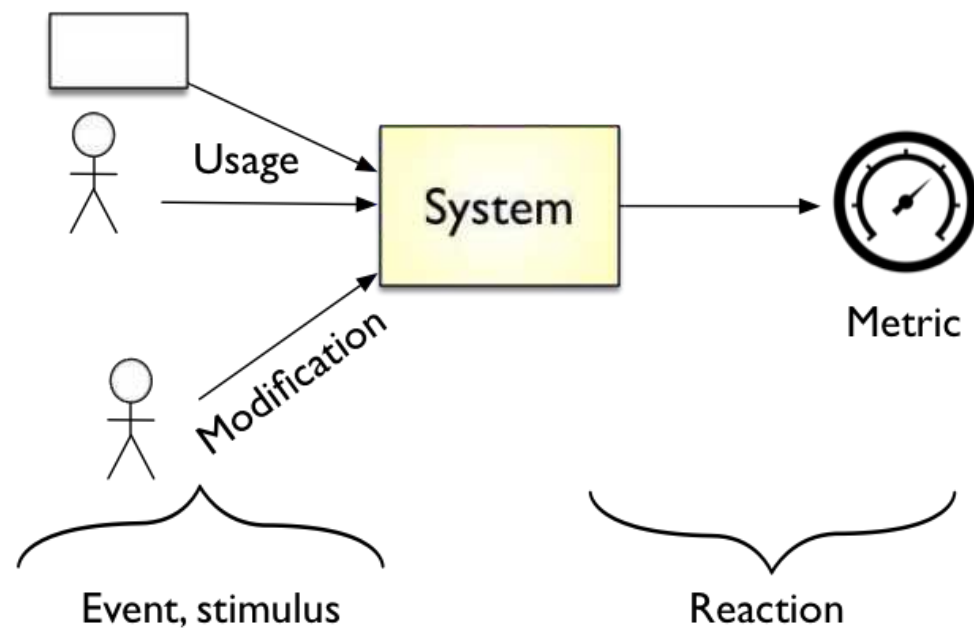


10.1 – 质量目标树

- 质量目标树的每个叶子节点代表一个质量属性场景
- 每个叶子有一个优先级
- 有时候会有很多的质量属性需求
- 格式
 - 思维导图，每个质量属性类别是一个分支
 - 每个质量属性类别有一个小节进行描述

10.2 质量属性场景

- 质量属性场景描述：当系统面对特定的刺激时，作出什么反应
- 2种类型
 - 使用：运行时反应
 - 用户请求1秒钟返回
 - 修改：对系统或者环境进行修改
 - 当面对一种用户类型时，必须增加一种新的用户类型
- 格式：表格或者文本



11 – 风险和技术债

- 已知技术风险或技术债务
 - 存在哪些潜在问题？
 - 开发团队对哪些方面感到困扰？
- 格式：
 - 列出风险或技术债务
 - 包含建议措施以最小化、缓解或避免风险，或减少技术债务

禁止复制

12 – 术语表

- 重要领域及技术术语
 - 利益相关方讨论系统时使用的术语
 - 通用词汇
 - 多语言环境下的翻译对照
 - 格式：表格

Term	Definition
...	...



03 复制 软件架构的检查工具

架构层面的代码检查工具

- 在代码中嵌入架构提示
 - 让读者能够从代码中推断出设计
 - 代码应反映架构
- 示例：
 - 将组件作为包
 - 将模块放置在不同的仓库/文件夹中
- 一些用于检查或强制执行架构约束的工具
 - <https://www.lattix.com/>, 依赖结构矩阵
 - <https://structurizr.com/>, 将系统设计用文本化方式定义, 并自动生成多层次图表

Structurizr: 用代码描述架构

```
workspace "Name" "Description"
```

```
!identifiers hierarchical
```

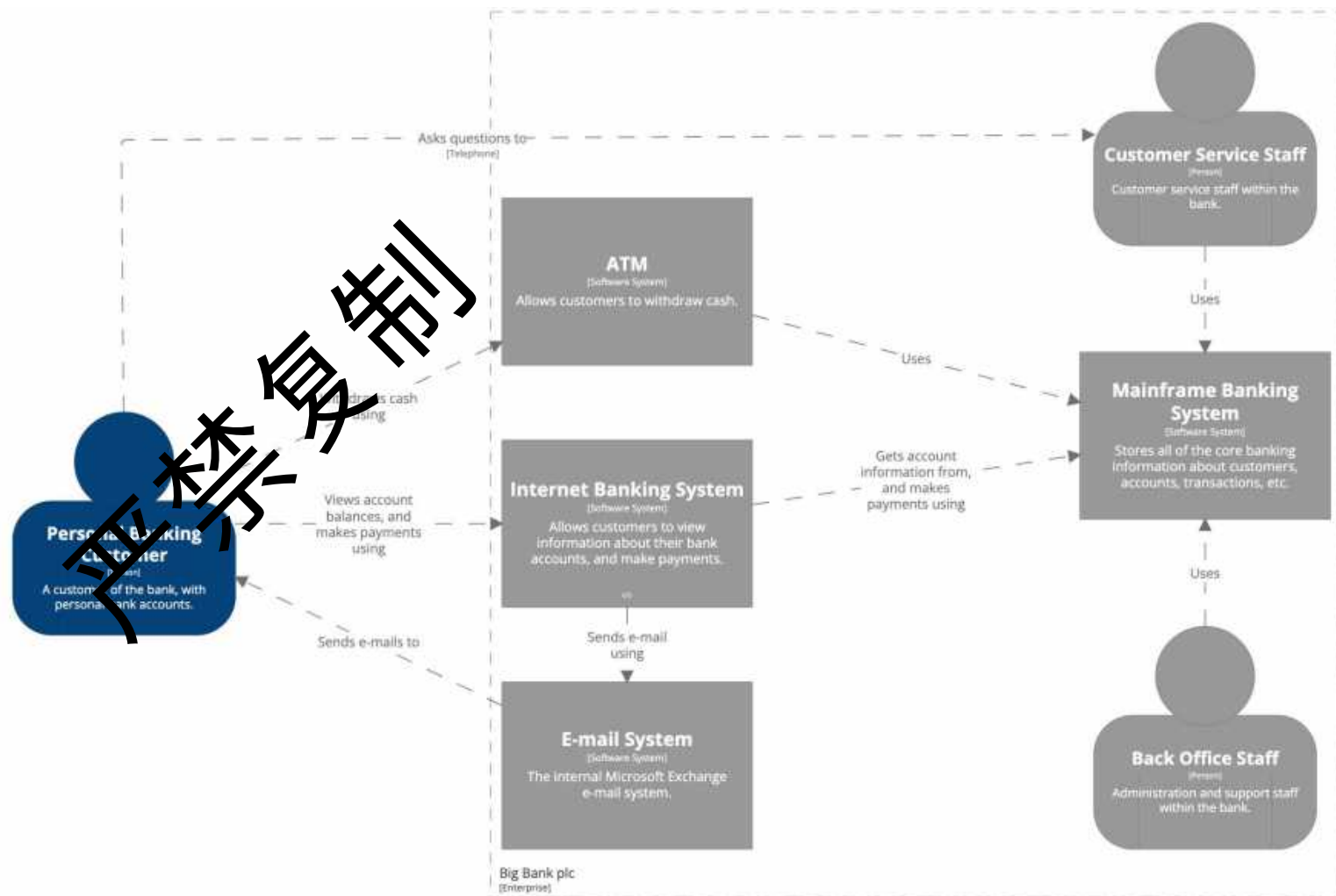
```
model {  
  u = person "User"  
  ss = softwareSystem "Software System" {  
    wa = container "Web Application"  
    db = container "Database Schema" {  
      tags "Database"  
    }  
  }  
  
  u -> ss "Uses"  
  u -> ss.wa "Uses"  
  ss.wa -> ss.db "Reads from and writes to"  
}
```

```
views {  
  systemContext ss "Diagram1" {  
    include *  
    autolayout lr  
  }  
}
```

```
  container ss "Diagram2" {  
    include *  
    autolayout lr  
  }  
}
```

```
// styling...
```

```
}
```



Archunit: 检查代码是否遵守架构规则

- 要求:

- Controller层 只能调用 Service层。
- Service层 只能调用 Repository层。
- Repository层 不能调用 Controller层。

```
// 加载项目的字节码
private final JavaClasses importedClasses = new ClassFileImporter()
    .importPackages("com.example");

@Test
public void controllerLayerShouldOnlyCallServiceLayer() {
    // 定义规则: Controller层只能调用Service层
    ArchRule rule = classes()
        .that().resideInAPackage("..controller..")
        .should().onlyDependOnClassesThat()
        .resideInAnyPackage("..service..", "java..", "javax..", "org.sp

    // 执行测试
    rule.check(importedClasses);
}
```

作业

- 选择赛车模拟系统中的一个模块，完成架构设计。
- 完成时间下次上课前（5-11）

严禁复制

THANK YOU

汇报人：WPS

严禁复制

